Week-5

Hema Harika

CH.SC.U4CSE24231

1.Quick Sort

Code

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```c
    int arr[n];
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    quickSort(arr, 0, n - 1);

    printf("Sorted array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n");
    return 0;

}
```

Output

```
hema-harika@hema-harika-VirtualBox:~/Downloads$ gcc quicksort.c -o quicksort
hema-harika@hema-harika-VirtualBox:~/Downloads$ ./quicksort
Enter number of elements: 5
Enter elements:
23
45
12
34
10
Sorted array:
10 12 23 34 45
```

2.Merge Sort

Code

```c
#include <stdio.h>

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
```

```c
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    mergeSort(arr, 0, n - 1);

    printf("Sorted array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
printf("\n");
    return 0;
}
```

Output

```
hema-harika@hema-harika-VirtualBox:~/Downloads$ gcc mergesort.c -o mergesort
hema-harika@hema-harika-VirtualBox:~/Downloads$ ./mergesort
Enter number of elements: 5
Enter elements:
12
56
34
67
44
Sorted array:
12 34 44 56 67
```

3.Binary Search Tree

Code

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};

struct node* newNode(int value) {
    struct node* temp = (struct node*)malloc(sizeof(struct node));
    temp->data = value;
    temp->left = temp->right = NULL;
    return temp;
}

struct node* insert(struct node* root, int value) {
    if (root == NULL)
        return newNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);

    return root;
}

void inorder(struct node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

```c
int main() {
    struct node* root = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter values:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("Inorder traversal of BST:\n");
    inorder(root);
printf("\n");
    return 0;
}
```

Output

```
hema-harika@hema-harika-VirtualBox:~/Downloads$ gcc binary.c -o binary
hema-harika@hema-harika-VirtualBox:~/Downloads$ ./binary
Enter number of nodes: 6
Enter values:
3
6
2
8
1
7
Inorder traversal of BST:
1 2 3 6 7 8
```